

PHP Syntax Core

- All examples assume **PHP 8.2+**
- Use `declare(strict_types=1);` for type checking.
- PHP syntax resembles **shell scripting**, but PHP is stricter and supports rich, object-oriented features shell scripts lack.

Variables & Types

```
<?php declare(strict_types=1);

$course = "ASE 230"; // string
$count  = 3;         // int
$ratio  = 3.14;       // float
$isOk    = true;      // bool
$nil     = null;      // null

// Dynamic reassignment is allowed but avoid mixing types.
$pi = 3.14159; // float
$pi = "3.14159"; // string (discouraged with strict typing mindset)
?>
```

- PHP variables start with \$
- Use typed params/returns/properties (next slides) to keep code safe.

IMPORTANT

PHP syntax is similar to shell scripting.

1. Each variable starts with , unlike many programming languages.
2. Variables are dynamically typed: their values can change type, which can cause hard-to-find bugs.

Constants & Enums

```
<?php declare(strict_types=1);

const APP_NAME = 'MyApp';
define('MAX_RETRY', 3); // legacy alternative

enum Status: string {
    case Draft = 'draft';
    case Published = 'published';
}
```

- Prefer const in code; define() exists for historical reasons.
- Enums (PHP 8.1+) replace ad-hoc string constants for state — invalid cases fail loudly instead of silently passing a typo.

```
// Old way: const/string, no type safety
const STATUS_DRAFT = 'draft';
function setStatus(string $status) { /* ... */ }
setStatus('drfat'); // typo compiles fine, fails silently at runtime

// New way: enum, type safety
enum Status: string {
    case Draft = 'draft';
}
function setStatusEnum(Status $status) { /* ... */ }
setStatusEnum(Status::Drfat); // fatal error when this line runs; no silent typo
```

- Prefer Enums over string constants for a fixed set of values.

```
// Create enum instance
$postStatus = Status::Draft;

// Compare
if ($postStatus === Status::Draft) {
    echo "Post is in draft mode\n";
}

// Switch / match
$message = match($postStatus) {
    Status::Draft => "Work in progress",
    Status::Published => "Publicly visible",
};

echo $message;    // "Work in progress"

// Access backing value
echo $postStatus->value;    // "draft"
```

Strings & Interpolation

- String interpolation is possible only with double quote (" ... ").

```
<?php declare(strict_types=1);

$name = "PHP";
echo "Hello $name\n";           // interpolation
echo 'Hello $name';             // literal; no interpolation

$heredoc = <<<TXT
Multi-line with $name
TXT;

$nowdoc = <<<'TXT'
Multi-line, no interpolation: $name
TXT;
```

Arrays (Indexed & Associative)

```
<?php declare(strict_types=1);

$colors = ['red', 'green', 'blue'];           // indexed
$user    = ['id' => 7, 'name' => 'Ada'];       // associative

$colors[] = 'purple';                         // push
[$first, $second] = $colors;                 // destructuring
$merged = [...$colors, 'black'];              // spread (7.4+)
?>
```

- Arrays unify lists & maps
- Use arrays for flexible lists or JSON-like data.

When to Use a DTO

Use a DTO (Data Transfer Object) when data has required fields and fixed types, such as a user received from an API.

- The constructor prevents missing or invalid values.

```
final class UserDto {  
    public function __construct(  
        public int $id,  
        public string $name,  
    ) {}  
}  
  
$user = new UserDto(id: 7, name: 'Ada');
```

IMPORTANT

Use objects, including DTO, for high-quality code.

- An array is not an object. Both hold multiple values, but DTOs enforce fields and types.
- DTOs make invalid data fail early.

```
// Array: missing fields are allowed  
$user = ['name' => 'Ada'];
```

```
// DTO: required fields and types are enforced  
$user = new UserDto(id: 'seven', name: 'Ada');  
// TypeError: id must be an int
```

Control Flow

| If Statement

```
<?php declare(strict_types=1);  
  
$score = 87;  
if ($score >= 90) {  
    $grade = 'A';  
} elseif ($score >= 80) {  
    $grade = 'B';  
} else {  
    $grade = 'C';  
}  
?>
```

| Ternary Operator

An if statement can be transformed into a ternary operator.

```
if ($score >= 60) {  
    $label = 'pass';  
}  
else {  
    $label = 'fail';  
}  
  
$label = ($score >= 60) ? 'pass' : 'fail'; // ternary
```

| Null Coalescing

The `??` operator is called the null-coalescing operator.

- It uses the left value when it exists and is not null; otherwise, it uses the right value.

```
$user = $_GET['u'] ?? 'guest';           // null coalescing
```

IMPORTANT

Null-related errors often appear only at runtime, making them harder to find and fix.

- Check nullable values before using them.
- Use `??` to provide a fallback value when a value is null.

Control Flow

Match Expression

```
<?php declare(strict_types=1);  
  
$ext = 'png';  
  
$mime = match ($ext) {  
    'png' => 'image/png',  
    'jpg', 'jpeg' => 'image/jpeg',  
    'gif' => 'image/gif',  
    default => 'application/octet-stream',  
};  
?>
```

- match is exhaustive and returns a value (no fall-through, safer than switch).

Loops

| while, for, and foreach

Use **while** to repeat while a condition is true, **for** for a counted loop, and **foreach** to visit each array item.

```
<?php declare(strict_types=1);

$i = 0;
while ($i < 3) { $i++; }

for ($j = 0; $j < 3; $j++) { /* ... */ }

$num = [10, 20, 30];
foreach ($num as $num) {
    echo $num;
}
?>
```

continue and break

`continue` skips the current iteration. `break` exits the loop completely.

```
$nums = [10, 20, 30];  
  
foreach ($nums as $num) {  
    if ($num === 20) continue; // skip 20  
    if ($num === 30) break;    // stop before 30  
    echo $num;                // prints 10  
}
```

Functions — Signatures & Returns

- PHP function gets arguments, processes the arguments, and returns.
- A function signature defines its name, parameter types, and return type.

```
<?php declare(strict_types=1);  
  
function add(int $a, int $b): int {  
    return $a + $b;  
}
```

Pass by Value

- The argument \$a is not changed.
- We call this "pass by value".

```
<?php declare(strict_types=1);  
  
// --- Pass by value (default)  
function incVal(int $x): void {  
    $x++;  
}  
$a = 1;  
incVal($a);  
echo $a; // 1 (unchanged)
```

Pass by Reference

Use `&` when a function must change the caller's variable.

```
function incRef(int &$x): void {  
    $x++;  
}  
  
$b = 1;  
incRef($b);  
echo $b; // 2 (changed)
```

IMPORTANT

- Pass by value is safe. Simple types such as int, float, and bool are copied as values.
- For large arrays and strings, PHP uses copy-on-write: it copies them only when they change.
- Pass by reference (`&`) lets a function change the caller's variable: Use it only when needed.
- Objects are passed by object handle, so their properties can change without `&`.

Copy-on-Write Example

```
$items = range(1, 1_000_000);

function countItems(array $items): int {
    return count($items);           // reads the shared array; no copy
}

function addItem(array $items): void {
    $items[] = 'new';               // creates a copy before changing it
}

addItem($items);
echo count($items);                 // 1000000: the original is unchanged
```

Variadic Arguments

- Use `...` when the number of arguments is unknown (variadic arguments).
- Inside the function, they are collected into an **array**.
- You can iterate, map, or reduce as needed.

```
// --- Variadic arguments
function sumAll(int ...$nums): int {
    return array_sum($nums);
}
echo sumAll(1, 2, 3, 4); // 10
?>
```

Named Function

A named function is declared once and called by its name.

- It may return a value or perform an action without **return**.

```
function triple(int $n): int {  
    return $n * 3;  
}  
  
echo triple(5); // 15  
  
function showTriple(int $n): void {  
    echo $n * 3;  
}  
  
showTriple(5); // 15; no return value
```

Closures

- A closure assigns a function to a variable.
- It uses the `function` keyword; use an explicit `return` when it returns a value.

```
<?php
$striple = function(int $n): int {
    return $n * 3;
};
echo $striple(5); // 15
?>
```

Arrow Functions

- An arrow function uses `fn` and `=>` for a one-line function.
- Its expression is returned automatically.

```
<?php declare(strict_types=1);  
  
$triple = fn(int $n): int => $n * 3;  
echo $triple(5); // 15  
?>
```

IMPORTANT

Modern programming prefers Closures or Arrow Functions.

- JavaScript (Node) and Python use them extensively.
- They let you pass behavior (functions) as a value, such as a callback or array operation.
- Use an arrow function for one expression; use a closure for multi-line logic.

Try It Yourself

- `module1/code/1_First_PHP_Server/syntax_demo.php` runs every example from this file in one script, in the same order, with printed output for each.
- Run it from the terminal: `php syntax_demo.php`.
- Read through it alongside this file, then try changing values and re-running it to see how the output changes.